

• INTERNAL ENGINEERING DOCUMENTATION

Seznik **POS** Project Documentation

Features, system architecture, request flows, implementation notes, deployment topology, and the testing & load-testing strategy behind the full-stack inventory & point-of-sale platform.

React 19 + TypeScript

Node.js + Express

PostgreSQL + Prisma

Vercel

AWS EC2

Web Bluetooth

k6 Load Testing

DOCUMENT

v1.0 — Engineering Reference

GENERATED

25 July 2026

REPOSITORY

Seznik-web (frontend/ + backend/)

LICENSE

ISC

Table of Contents

01	Product Overview	10	Deployment — Backend & Database on AWS
02	Feature Breakdown	11	Testing & Quality Assurance Strategy
03	System Architecture	12	API Testing — Frontend & Backend
04	Frontend Architecture & Folder Structure	13	Load & Stress Testing
05	Backend Architecture & Folder Structure	14	Security Considerations
06	End-to-End Request Flows	15	Theming, Accessibility & Internationalization
07	How Key Features Were Implemented	16	Roadmap & Recommended Next Steps
08	Libraries & Tools Matrix	A	Appendix — Data Model, Routes & Env Vars
09	Deployment — Frontend on Vercel		

SECTION 01

Product Overview

Seznik POS is a full-stack Point of Sale & Inventory Management platform for retail stores. It bundles inventory tracking, billing/checkout, Bluetooth thermal-printer & barcode-label output, sales/purchase/expense/credit ledgers, and reporting into one product, with every business's data isolated by account (userId-scoped) inside a single shared PostgreSQL database.

13

PRISMA DATA MODELS

11

BACKEND ROUTE
MODULES

15

FRONTEND DOMAIN
PAGES

15

REACT QUERY HOOKS

• Who it's for

Store Owners / Admins

Full access: inventory, pricing, reports, staff accounts, printer & invoice configuration.

Staff / Agents

Sub-accounts (ManagedUser) scoped by role-based permission flags — e.g. stock edit, purchases, reports.

Counter / Checkout Use

A dedicated fast POS and a mobile-first "POS-Lite" mode for high-volume, on-the-go billing.

• Two independently deployed applications

The codebase is a two-folder workspace — frontend/ (React SPA) and backend/ (Express REST API) — with **no shared code or types** between them. They only ever communicate over HTTPS/JSON, authenticated by a bearer JWT. This keeps the two halves independently deployable (Vercel vs. AWS) and independently scalable.

SECTION 02

Feature Breakdown

● Authentication & Access

OTP Email Verification Signup

6-digit code, 10-minute expiry, 60s resend cooldown, 5 max attempts, hashed at rest with bcrypt before a 30-minute "verified" window unlocks registration.

JWT Sessions

Stateless bearer tokens (30-day expiry) attached by a shared fetchApi wrapper; no server-side session store.

Admin + Staff Sub-Accounts

User (owner/admin) and ManagedUser (agent) share one login-by-email space; an email can never back two accounts across either table.

Workstation Access Selection

Re-confirms the account password before switching role context on a shared/kiosk device.

Forgot Password (OTP)

4-step reset flow (request → verify → set new password → confirmation) mirroring the signup OTP mechanics.

Guided Onboarding

Business-profile setup + first category creation before a new owner reaches the dashboard.

● Inventory & Categories

Product Catalog

Grid/list views, image upload, per-product barcode, low-stock threshold, unit type, bulk delete, barcode-driven stock updates.

Hierarchical Categories

One level of subcategories (e.g. "Shirts" → "Checks", "Plain") via a self-referential parentId on Category; expandable tree UI with per-branch product counts.

● Point of Sale

POS & POS-Lite

Card-grid product picker, live search (name / SKU / barcode / price), a premium filter panel (stock status, price range, sort), and category pills including subcategories.

Cart & Checkout

Quantity steppers, order-level flat/% discount, live tax computation, four payment methods (cash / card / UPI / credit) with change calculation.

Barcode Scanning

Hardware-scanner listener plus a manual scan-entry mode for scan-to-sell and scan-to-restock.

Receipts, 3 Ways

A4 browser print, 50/80mm thermal via the system print dialog, or a true Bluetooth ESC/POS print straight to a paired thermal printer.

● Printers & Label Design

Web Bluetooth Printer Pairing

GATT-based pairing with auto-reconnect for known devices (Seznik Dev / Veer) and one-click gap-sensor calibration.

Interactive Label Designer

Drag/align/bold/scale layout builder bound to real product data; generates Code128, EAN-13, and QR symbologies.

Dual Protocol Output

TSPL (gap-sensing label rolls) and ESC/POS (compact thermal receipt rolls) encoders, plus a browser/system-driver fallback.

Invoice Themes

A4 invoices in four color themes with terms & conditions and a UPI payment QR code.

● Business Operations & Reporting

Sales • Purchases • Expenses • Credits

Full CRUD with detail pages; credit transactions maintain a running customer ledger.

Reports

Profit & loss, tax report, sales report, top products/categories, payment-mode & profit-breakdown charts, expense summary — each its own aggregation endpoint.

● Platform & Experience

Installable PWA

Manifest + service worker via vite-plugin-pwa for an installable, app-like frontend.

Dark / Light Theme

App-wide ThemeContext with Tailwind dark: variants on every screen.

Multi-Language

LanguageContext-driven i18n layer for UI copy.

First-Run Product Tours

A DOM-spotlighting interactive tour per page that auto-fires **once** on first visit, plus an on-demand "Video Guide" modal.

Marketing Landing Page

Three.js / React Three Fiber 3D scenes with Framer Motion scroll animations (currently built, not yet linked into routing).

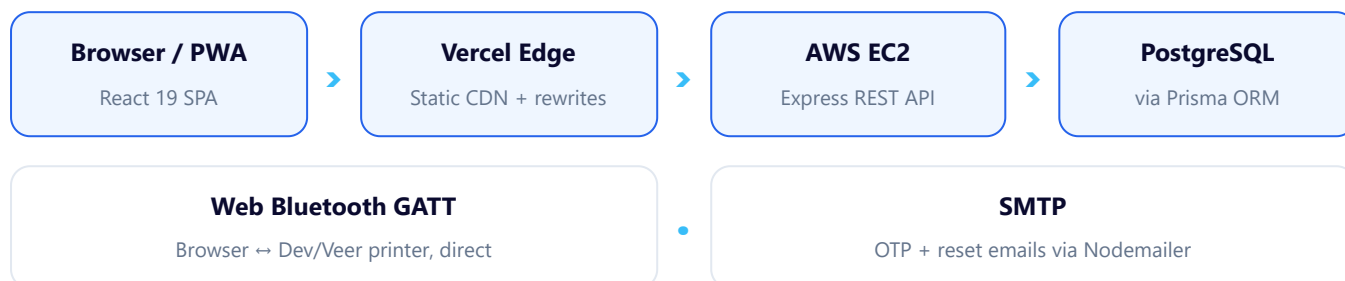
Help Chatbot

FAQ-driven assistant widget available across the authenticated app.

SECTION 03

System Architecture

A classic three-tier architecture with one deliberately unusual path: the browser talks **directly** to Bluetooth hardware for the physical print step, bypassing the backend entirely once it has the data it needs.



Design principles

Stateless API

No server-side sessions — all auth state is a signed JWT the client stores and replays; any EC2 instance running the API is interchangeable.

Single source of truth for data

The frontend never talks to PostgreSQL directly; every read/write goes through Prisma on the backend.

Tenant isolation by row

Every business table carries a `userId` foreign key; queries are always scoped to the authenticated user's own rows.

Hardware stays client-side

ESC/POS & TSPL byte streams are built in the browser and written to the printer's GATT characteristic — no server round-trip for the physical print.

SECTION 04

Frontend Architecture & Folder Structure

React 19 + TypeScript, bundled by Vite, routed with React Router v7 using lazy, route-level code splitting. Server state lives in TanStack Query; cross-cutting concerns (auth, theme, language) live in React Contexts; everything else is local component state.

● Layering

Pages

One folder per domain under `pages/` (products, pos, categories, sales...), composed from shared UI primitives.

Hooks

`hooks/useX.ts` wraps TanStack Query around a matching service — caching, invalidation-on-mutation, background refetch.

Services

`services/xService.ts` — one per domain — call the shared `fetchApi()` wrapper that attaches the JWT and normalizes errors.

Folder structure — frontend/src

```
src/
├── assets/      # product/printer imagery, logos
├── components/
│   ├── ui/     # Button, Card, Modal, Input, Select, Badge, Tabs...
│   ├── layout/ # AppLayout, Sidebar, Topbar, MobileNav, PrivateRoute
│   ├── forms/  # ImageUpload, DateRangePicker
│   ├── data-display/ # DataTable, StatsCard, StatusBadge
│   └── common/  # InteractivePageTour, PageVideoTutorialModal, HelpChatBot
├── contexts/    # AuthContext, ThemeContext, LanguageContext
├── hooks/       # useProducts, useSales, useCart, useBlePrinter, usePageTutorial...
├── services/    # api.ts + one *Service.ts per domain
├── pages/       # auth, dashboard, pos, products, categories, sales,
│               # purchases, expenses, credits, customers, suppliers,
│               # reports, settings, printers, landing
├── types/       # hand-written interfaces mirroring the Prisma models
├── utils/       # receipt.ts, labelPrint.ts, blePrinter.ts, categoryTree.ts...
├── constants/   # routes.ts, queryKeys.ts, units.ts
├── i18n/        # translation tables for LanguageContext
└── data/        # chatbot FAQ + page-tutorial content
```

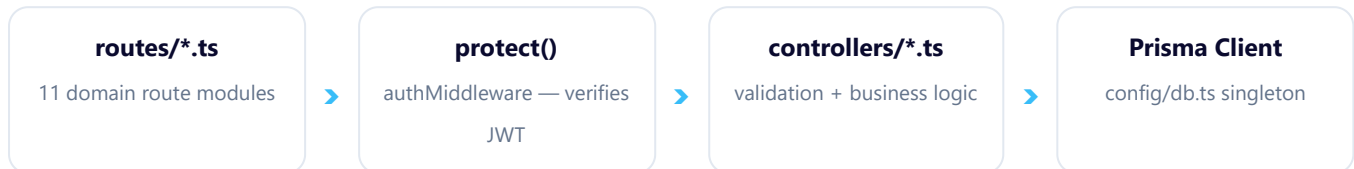
● Key libraries at this layer

React Router v7 (routing), TanStack Query (server state), react-hook-form + Zod (forms & validation), Tailwind CSS (styling), Recharts (charts), Lucide (icons), Framer Motion + three.js/@react-three (landing page motion & 3D), react-hot-toast (notifications).

SECTION 05

Backend Architecture & Folder Structure

A layered Express API — **Routes** → **Middleware** → **Controllers** → **Prisma** → **PostgreSQL** — with one route file and one controller file per domain, and cross-cutting auth handled by a single middleware.



Folder structure — backend/src

```

src/
├── app.ts      # Express app: CORS, JSON body parsing, route mounting, /health
├── server.ts   # app.listen(PORT) — process endpoint
├── config/
│   └── db.ts   # PrismaClient singleton
├── routes/     # auth, category, settings, customer, supplier, product,
│               # sale, purchase, expense, credit, report
├── controllers/ # one controller per domain (11 files)
├── middlewares/
│   └── authMiddleware.ts # protect() — verifies JWT, attaches req.user
├── services/
│   └── emailService.ts  # Nodemailer/SMTP wrapper (OTP + reset emails)
├── utils/
│   └── jwt.ts           # token sign/verify helpers

prisma/
├── schema.prisma # 13 models, PostgreSQL datasource
├── migrations/

tests/           # runPreDeploymentSuite.js, stressTest.js, stress-test.k6.js, testEmail.js
  
```

● API surface

Every route is mounted under `/api/*` in `app.ts`: `/auth`, `/categories`, `/settings`, `/customers`, `/suppliers`, `/products`, `/sales`, `/purchases`, `/expenses`, `/credits`, `/reports` — plus an unauthenticated `/health` check used by load tests and uptime monitoring.

● Data model at a glance

13 Prisma models: **User**, **EmailOtp**, **ManagedUser**, **Category** (self-referential `parentId` for one level of subcategories), **Settings**, **Customer**, **Supplier**, **Product**, **StockHistory**, **Sale**, **Purchase**, **Expense**, **CreditTransaction**. See the Appendix for the full field-level reference.

SECTION 06

End-to-End Request Flows

Two representative flows, traced through every layer — a standard authenticated write, and the security-relevant OTP signup path.

• Flow A — "Complete & Print" a sale in POS

- 1 User clicks Complete & Print** **FRONTEND**
POSPage.tsx builds the sale payload from cart state.
- 2 useCreateSale() mutation fires** **FRONTEND**
TanStack Query hook (hooks/useSales.ts) calls saleService.createSale().
- 3 fetchApi() sends the request** **FRONTEND**
Attaches Authorization: Bearer <JWT> from localStorage, POSTs JSON to VITE_API_URL + /sales.
- 4 Vercel rewrite proxies /api/*** **EDGE**
The browser only ever sees one origin; vercel.json forwards to the AWS-hosted API.
- 5 Express routes the request** **BACKEND**
saleRoutes.ts → protect() middleware verifies the JWT and attaches req.user.
- 6 saleController.createSale runs** **BACKEND**
Validates the payload, computes totals server-side, decrements product stock.
- 7 Prisma writes to PostgreSQL** **DATABASE**
Sale + StockHistory rows are written inside the same request, scoped to req.user.id.
- 8 201 response bubbles back** **BACKEND**
Returns the created sale's id and invoice number.
- 9 Cache invalidation & UI update** **FRONTEND**
onSuccess invalidates the sales/products query caches, clears the cart, opens the print-format modal.
- 10 Bluetooth print (optional)** **HARDWARE**
If "Print via Bluetooth" is chosen, the browser encodes ESC/POS bytes and writes them directly to the paired printer's GATT characteristic — no backend round-trip.

• Flow B — OTP-gated signup

- 1 Submit email** **FRONTEND**
LoginPage → POST /auth/send-otp.
- 2 Uniqueness check** **BACKEND**
findAccountByEmail() checks BOTH the User and ManagedUser tables before issuing a code.
- 3 Code generated & hashed** **BACKEND**
6-digit OTP, bcrypt-hashed, stored with a 10-minute expiry.
- 4 Email dispatched** **SMTP**
Nodemailer sends the code via the configured SMTP provider.
- 5 User enters the code** **FRONTEND**
POST /auth/verify-otp — 5 attempts max before logout.
- 6 30-minute verified window opens** **BACKEND**
Registration is only accepted while this window is open.
- 7 POST /auth/register** **BACKEND**
Re-checks uniqueness, hashes the password with bcrypt, creates the User row.
- 8 JWT issued** **FRONTEND**
Token stored client-side; user proceeds into onboarding.

SECTION 07

How Key Features Were Implemented

● Bluetooth thermal printing

Built on the **Web Bluetooth API** (`hooks/useBlePrinter.ts`, `utils/blePrinter.ts`): the browser requests a GATT connection to the printer, discovers its writable characteristic, and streams raw command bytes to it. Two encoders live in `utils/receipt.ts` / `utils/labelPrint.ts` — an **ESC/POS** encoder for thermal receipts and a **TSPL** encoder (with GAP/SIZE commands) for gap-sensed label rolls — so the same UI can drive two different printer languages. A system-driver/browser-print path is kept as a fallback for non-Bluetooth printers.

● Category hierarchy

Subcategories reuse the existing Category table via a single self-relation (`parentId` → `Category.id`, cascading delete), capped at one level deep by validation in the controller rather than the schema. Every dropdown, filter, and pill list in the app builds its options through one shared helper (`utils/categoryTree.ts`) so the parent/child ordering is consistent everywhere categories are surfaced.

● Authentication & email uniqueness

Passwords are hashed with **bcrypt**; sessions are stateless **JWTs** (jsonwebtoken). Because staff sub-accounts (ManagedUser) and primary accounts (User) share one login-by-email space, every account-creation path — signup, social login, and "Add User" in Settings — routes through a single `findAccountByEmail()` helper that checks both tables before allowing a new account.

● First-run product tours

InteractivePageTour spotlights real DOM elements (via `data-tour` attributes and `getBoundingClientRect`) with an auto-positioned speech-bubble popover. `usePageTutorial.ts` gates the auto-trigger with a per-page `localStorage` flag written the instant the tour opens — not when it's dismissed — so navigating away mid-tour can never make it reappear. A separate "Video Guide" modal (`PageVideoTutorialModal`) lets a user replay it manually at any time, unaffected by that flag.

● 3D / animated landing page

Built with **three.js** via **@react-three/fiber** and **@react-three/drei** for ambient particle fields and floating geometry synced to scroll position, and **Framer Motion** for scroll-triggered reveals, parallax, and mouse-tilt product shots (the Dev & Veer printers). Currently implemented but intentionally not wired into the router for this release.

SECTION 08

Libraries & Tools Matrix

Frontend

LIBRARY	LAYER	PURPOSE
react / react-dom	Core	UI runtime (v19)
vite	Build	Dev server & production bundler
typescript	Language	Static typing across the SPA
react-router-dom	Routing	v7 client-side routing, lazy route splitting
@tanstack/react-query	Data	Server-state cache, mutations, invalidation
@tanstack/react-virtual	Data	Virtualized long lists
react-hook-form + zod	Forms	Form state + schema validation
tailwindcss	Styling	Utility-first CSS, dark-mode variants
recharts	UI	Dashboard & report charts
lucide-react	UI	Icon set
react-hot-toast	UI	Toast notifications
three / @react-three/fiber / drei	Landing page	3D scenes & scroll-linked geometry
framer-motion	Landing page	Scroll/entrance animation
xlsx	Utility	Excel export for reports
react-to-print	Utility	A4 invoice printing
browser-image-compression	Utility	Client-side image compression before upload
vite-plugin-pwa	Platform	Installable PWA manifest & service worker
vitest / @testing-library/*	Testing	Unit/component test infrastructure

Backend

LIBRARY	LAYER	PURPOSE
express	Core	HTTP server & routing
typescript / tsx	Language	Typed backend, dev hot-reload via tsx watch
@prisma/client + prisma	Data	Type-safe ORM & migrations against PostgreSQL
bcrypt / bcryptjs	Security	Password & OTP hashing
jsonwebtoken	Security	Stateless session tokens

LIBRARY	LAYER	PURPOSE
<code>cors</code>	Security	Cross-origin request handling
<code>nodemailer</code>	Integration	SMTP delivery for OTP & reset emails
<code>dotenv</code>	Config	Environment variable loading
<code>date-fns</code>	Utility	Date math for reports & OTP windows

Testing & Ops Tooling

`k6` load testing · `Node.js http module` bespoke stress scripts · `Vitest` + `React Testing Library` component tests · `ESLint` + `tsc --noEmit` static analysis gates · `Prisma Studio` ad-hoc DB inspection.

SECTION 09

Deployment — Frontend on Vercel

The frontend ships as a static build to Vercel's global CDN, connected to the Git repository for build-on-push deploys.

1 Build

`npm run build` runs `tsc -b` (type-check) then `vite build`, emitting a static `dist/` bundle.

2 Deploy to Vercel

Vercel builds and publishes `dist/` to its edge network on every push to the connected branch.

3 API rewrite

`vercel.json` rewrites `/api/:path*` to the AWS-hosted Express API, so the browser only ever calls one origin — no CORS friction, backend host stays out of the client bundle's URLs.

4 SPA fallback

A catch-all rewrite (`/(.*) → /index.html`) so React Router's client-side routes survive a hard refresh or deep link.

5 Build-time config

`VITE_API_URL` is baked into the bundle at build time to point at the production API base path.

frontend/vercel.json

```
{
  "rewrites": [
    { "source": "/api/:path*", "destination": "http://<aws-ec2-host>:5000/api/:path*" },
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}
```

The PWA manifest and service worker (via `vite-plugin-pwa`) ship in the same build, so the deployed app is installable straight from the browser.

SECTION 10

Deployment — Backend & Database on AWS

The API and its database run on AWS, reachable at a fixed public address that Vercel's rewrite forwards to.

- 1

EC2 instance
Hosts the Express API, with the security group opened for inbound traffic on the API port (5000) and SSH (22) for administration.
- 2

Build
`npm run build` compiles TypeScript (tsc) to dist/.
- 3

Run
`npm start` (`node dist/server.js`) is kept alive on the instance — e.g. via PM2 or a systemd unit — so the API survives reboots and auto-restarts on crash.
- 4

Database
PostgreSQL, reached over a TLS connection (`sslmode=require`) via `DATABASE_URL`, hosted on AWS alongside the API instance.
- 5

Schema deploy
Prisma applies schema changes against the AWS database (`prisma migrate deploy` / `prisma db push` for additive dev changes), then `prisma generate` regenerates the typed client the API imports.
- 6

Health check
The unauthenticated GET `/health` endpoint gives load balancers / uptime monitors a cheap liveness probe.

Production environment variables

VARIABLE	PURPOSE
PORT	API listen port (default 5000)
NODE_ENV	production / development switch (also toggles dev-mode OTP console logging)
DATABASE_URL	PostgreSQL connection string (TLS required)
JWT_SECRET	Signing key for session tokens
CORS_ORIGIN	Allowed frontend origin(s)
SMTP_HOST / PORT / USER / PASS / FROM	Outbound email for OTP & password-reset delivery

SECTION 11

Testing & Quality Assurance Strategy

Quality is enforced through a mix of static analysis gates run continuously during development, and scripted checks run against a live, running API before a change ships.

Type-checking as a correctness gate

Both apps are built with `tsc` in strict mode. Because the two apps share no code, hand-written frontend interfaces are kept in lockstep with the Prisma schema by convention, and a full `tsc -b --noEmit` is run after every change.

Linting

ESLint (`npm run lint`) enforces React Hooks rules and general code-quality on the frontend.

Component test infrastructure

Vitest + React Testing Library + `jsdom` (`src/test/setup.ts`) are configured and ready; this is the layer to grow as coverage expands.

Manual functional QA

Every feature is exercised against the real running dev servers (both apps live) before being considered complete — including edge cases like duplicate emails, stock limits, and dark-mode contrast.

See Sections 12–13 for how the backend's actual HTTP surface and database are tested end-to-end and under load.

SECTION 12

API Testing — Frontend & Backend

● Backend — pre-deployment smoke suite

`backend/tests/runPreDeploymentSuite.js` is a dependency-free Node script (built on the core `http` module) that drives the **real, running API** end-to-end: it logs in, then exercises `auth`, `categories`, `products`, `sales`, `purchases`, `expenses`, and `reports`, recording a **pass/fail + latency** line per check with a colored console report. It's run manually with `node tests/runPreDeploymentSuite.js` before a deploy, against either `localhost` or a staging `TEST_BASE_URL`.

● Backend — SMTP sanity check

`backend/tests/testEmail.js` sends a real test email through the configured SMTP provider, confirming OTP and password-reset delivery actually works before relying on it in production.

● Frontend — served-app smoke/stress pass

`frontend/scripts/stress-test-frontend.js` (`npm run test:stress`) requests every major route (`/`, `/login`, `/dashboard`, `/pos`, `/products`, `/sales`, `/expenses`, `/customers`, `/suppliers`, `/reports`) against the locally served build, confirming the SPA serves cleanly under concurrent load before it ever reaches Vercel.

Together these three scripts answer: "does the API actually work end-to-end?", "does outbound email actually work?", and "does the built app actually serve under load?" — the three questions a type-checker can't answer on its own.

SECTION 13

Load & Stress Testing

• k6 — staged virtual-user ramp

backend/tests/stress-test.k6.js, run with `npm run test:k6` (k6 run), authenticates once in a `setup()` phase and reuses that JWT across every virtual user. It ramps **10 → 50 → 100 concurrent VUs** over ~70 seconds against `/health`, `/products`, `/categories`, `/sales`, `/expenses`, `/reports/dashboard`, and `/reports/p1`, with explicit pass/fail SLA thresholds:

p95 < 500ms

95TH PERCENTILE LATENCY

p99 < 1000ms

99TH PERCENTILE LATENCY

< 1%

ERROR RATE

100 VUs

PEAK CONCURRENCY

• Node concurrency stress test

backend/tests/stressTest.js drives **50 concurrent workers** through **1,000 real CRUD operations** directly against the live API and database — creating/reading products, categories, and expenses — to surface connection-pool exhaustion or write-contention issues that a read-only load test wouldn't catch.

• Frontend concurrent page-load test

frontend/scripts/stress-test-frontend.js fires **50 concurrent clients** across **1,000 total requests** spread over every top-level route, checking the served SPA holds up before it's deployed.

• Tooling

k6 for scripted, threshold-driven load tests with staged ramps — the industry-standard tool for this.

Plain Node.js (http module) for bespoke concurrency/CRUD stress scripts, avoiding an extra dependency. Manual verification in the browser closes the loop on anything a script can't judge (visual regressions, animation smoothness, hardware pairing UX).

SECTION 14 • ADDITIONAL

Security Considerations

Password & OTP hashing

bcrypt for both account passwords and OTP codes at rest — a leaked database never exposes a usable plaintext code or password.

OTP abuse controls

10-minute code expiry, 60-second resend cooldown, and a 5-attempt lockout before a fresh code is required.

Stateless JWT sessions

30-day signed tokens; no server-side session store to leak or desync.

Cross-account email uniqueness

One email can never back two accounts (owner or staff) — enforced centrally through `findAccountByEmail()` on every account-creation path.

Row-level tenant isolation

Every business table carries `userId`; controllers always scope reads/writes to `req.user.id` from the verified JWT.

TLS to the database

`DATABASE_URL` requires `sslmode=require` for the Prisma connection to PostgreSQL.

SECTION 15 • ADDITIONAL

Theming, Accessibility & Internationalization

Dark / Light Theme

App-wide ThemeContext paired with Tailwind's dark: variant on every screen and shared UI primitive — not a bolt-on, it's baked into the design system.

Internationalization

LanguageContext drives a translation-table layer (i18n/translations) so UI copy can switch languages without touching component logic.

Mobile-first checkout

A dedicated "POS-Lite" mode and responsive card-grid layouts (down to a stacked mobile POS product picker) for counter devices of any size.

Guided discovery

Once-only first-run spotlighted tours plus always-available on-demand video guides reduce onboarding friction without becoming an annoyance on repeat visits.

SECTION 16 • ADDITIONAL

Roadmap & Recommended Next Steps

Ship the landing page

The Three.js/Framer Motion marketing site is built and ready — wiring it back into routing is a scoping decision, not an engineering blocker.

Native Android & iOS apps

Already teased on the landing page's "Coming Soon" section — a natural extension once the web PWA's feature set stabilizes.

Automate the test suites

The pre-deployment suite, k6 load test, and stress scripts all currently run manually — wiring them into a CI/CD pipeline (e.g. GitHub Actions on push) would catch regressions before merge instead of before deploy.

Expand component test coverage

Vitest + React Testing Library are configured but largely unused today — the highest-leverage next addition given how much UI logic (cart totals, filters, category trees) lives client-side.

APPENDIX

Data Model, Routes & Environment Reference

Prisma models

UserEmailOtpManagedUserCategorySettingsCustomerSupplierProductStockHistorySalePurchaseExpenseCreditTransaction

REST API modules (all under /api)

BASE PATH	AUTH	COVERS
/auth	Mixed	OTP signup, login, social login, forgot-password, profile, role, onboarding, managed users
/categories	JWT	Category + subcategory CRUD, active toggle
/products	JWT	Product catalog CRUD, barcode lookup, stock
/customers	JWT	Customer CRUD
/suppliers	JWT	Supplier CRUD
/sales	JWT	Sale creation & history
/purchases	JWT	Purchase order CRUD
/expenses	JWT	Expense CRUD
/credits	JWT	Customer credit ledger
/settings	JWT	Business profile, invoice/printer config
/reports	JWT	Dashboard, P&L, tax, top products/categories, payment modes
/health	Public	Liveness probe

Full environment variable reference

APP	VARIABLE	NOTES
Backend	PORT	Default 5000
Backend	NODE_ENV	production / development
Backend	DATABASE_URL	postgresql://...?sslmode=require
Backend	JWT_SECRET	Session token signing key
Backend	CORS_ORIGIN	Allowed frontend origin
Backend	SMTP_HOST/PORT/USER/PASS/FROM	OTP & reset email delivery
Frontend	VITE_API_URL	Production API base URL, baked in at build time

Seznik POS — Internal Engineering Documentation · Generated from the live codebase on 25 July 2026 · For internal & stakeholder reference.